

BED-1304 Python-Lab/ SOK-3091 Python for økonomer

Oppgaveark: Grunnleggende koding og dataanalyse

Handelshøgskolen ved UiT

Høsten 2026

Introduksjon

Dette oppgavearket er designet for å repetere grunnleggende programmering i Python for økonomistudenter. Fokuset er på variabler, funksjoner, datastrukturer (lister, tupler, dictionaries), samt bruk av bibliotekene **numpy** og **pandas**. Helt til slutt i dokumentet finner du løsningsforslag (fasit) for alle oppgavene.

Når du jobber med Pandas-oppgaven, skal du ta utgangspunkt i datasettet `Median_totalpris_boliger_Oslo.csv`. Den finner dere på nettsiden til kurset.

1 Python Basics og Variabler

Når vi jobber med økonomiske modeller, er det viktig å kunne definere og regne med variabler.

Oppgave 1.1 Opprett en variabel `inntekt` med verdien 650000 og en variabel `skattesats` med verdien 0.25. Regn ut hvor mye som skal betales i skatt, og lagre dette i en ny variabel `skattetrekk`.

Oppgave 1.2 Bruk en f-streng for å skrive ut resultatet på en ryddig måte. Utskriften skal være:

"Med en bruttoinntekt på 650000 kr og en skattesats på 25%, blir skattetrekket 162500.0 kr."

Oppgave 1.3: Forklar koden Hva blir resultatet (output) av følgende kode, og hvorfor?

```
1 overskudd = 50000
2 overskudd = overskudd - 10000
3 print(overskudd)
```

2 Funksjoner

Økonomer bruker ofte funksjoner for beregninger som skal gjøres mange ganger, for eksempel for å finne nåverdi (Present Value). Formelen for nåverdi er:

$$PV = \frac{FV}{(1 + r)^n}$$

Oppgave 2.1 Lag en funksjon `beregn_naaverdi(fremtidig_belop, rente, aar)` som tar inn tre parametere og returnerer nåverdien.

Oppgave 2.2 Test funksjonen din. Anta at du forventer å motta 150 000 kr om 5 år, og at avkastningskravet (renten) er 8% (0.08). Hva er nåverdien? Bruk `round()` for å avrunde til to desimaler ved utskrift.

Oppgave 2.3: Forklar koden En kollega har skrevet følgende funksjon for å regne ut merverdiavgift (MVA):

```
1 def beregn_mva(pris):
2     mva = pris * 0.25
3     print(mva)
4
5 totalpris = 1000 + beregn_mva(1000)
```

Når koden kjøres, krasjer programmet med en `TypeError`. Forklar nøyaktig hvorfor dette skjer, og hvordan du vil rette feilen.

3 Datastrukturer (Lister, Tupler, Dictionaries) og NumPy

Når vi har mye data, trenger vi gode strukturer for å lagre og manipulere dem effektivt.

Oppgave 3.1: Dictionaries Lag et oppslagsverk (dictionary) kalt `aksjer` som inneholder følgende selskaper (ticker) og deres tilhørende aksjekurs: EQNR = 290, DNB = 210, YAR = 350. Skriv deretter kode for å legge til et nytt selskap, MOWI, med kursen 190.

Oppgave 3.2: Lister og Tupler Du har en liste med historiske kontantstrømmer: `kontantstrommer = [-1000, 200, 300, 400, 500]`. Hent ut (ved hjelp av indeksering) den opprinnelige investeringen (det første tallet), og lagre den i en variabel. Forklar også kort (for deg selv) hvorfor man noen ganger ville brukt en Tuple i stedet for en Liste til slike data.

Oppgave 3.3: NumPy Vi ønsker å oppjustere alle de positive kontantstrømmene i listen over med 5% på grunn av inflasjon. Importer `numpy` som `np`. Gjør listen om til et NumPy-array. Multipliser hele arrayet med 1.05. Skriv ut det nye arrayet.

Oppgave 3.4: Forklar koden Hva er forskjellen på output fra Kodeblokk A og Kodeblokk B under? Forklar resultatene.

```
1 # Kodeblokk A
2 priser_liste = [100, 200, 300]
3 print(priser_liste * 2)
4
5 # Kodeblokk B
6 import numpy as np
7 priser_array = np.array([100, 200, 300])
8 print(priser_array * 2)
```

4 Pandas og databehandling

I denne delen skal vi bruke datasettet for boligpriser i Oslo, kalt `Median_totalpris_boliger_Oslo.csv`. Det inneholder median totalpris for boliger fordelt på bydeler og år.

Oppgave 4.1 Importer `pandas` som `pd`. Les inn csv-filen `Median_totalpris_boliger_Oslo.csv` og lagre den i en variabel `df`. Vis de 5 første radene.

Oppgave 4.2 Hvor mange rader og kolonner har datasettet? Skriv koden for å finne dette, og sjekk samtidig om datasettet inneholder noen manglende verdier (NaN).

Oppgave 4.3 Lag et uttrekk av datasettet som kun inneholder kolonnen 'Geografi' og kolonnen for året "2023". Sett 'Geografi' som index.

Oppgave 4.4 Filtrer dataene slik at du kun henter ut raden for 'Bydel Frogner'. Skriv ut hva medianprisen var i Frogner dette året.

Oppgave 4.5: Forklar koden Gitt det innleste datasettet med boligpriser, hva gjør nøyaktig følgende kodelinje?

```
1 df_topp = df.sort_values(by="2023", ascending=False).head(3)
```

5 Utfordring: Simulering av investering

I finans og økonomi vet vi sjelden nøyaktig hva fremtiden bringer. Derfor bruker vi ofte simuleringer for å modellere usikkerhet.

Oppgave 5.1 (Challenge) Du har 100 000 kr å investere i dag. Hvert år er den årlige avkastningen tilfeldig, og varierer jevnt mellom et fall på 2% (-0.02) og en oppgang på 8% (0.08).

Skriv et program som bruker en `while`-løkke for å finne ut hvor mange år det tar før investeringen din har doblet seg (altså passert 200 000 kr). Bruk `np.random.uniform(-0.02, 0.08)` inni løkken for å trekke den årlige avkastningen. Skriv ut antall år det tok til slutt ved hjelp av en f-streng.

NB: Her bruker vi ting fra hele pensum, så dette er ment som en nøtt for dem som vil bryne seg!

Løsningsforslag (Fasit)

Under finner du forslag til hvordan oppgavene kan løses, samt de konseptuelle forklaringene.

Del 1: Python Basics og Variabler

```
1 # Oppgave 1.1
2 inntekt = 650000
3 skattesats = 0.25
4 skattetrekk = inntekt * skattesats
5
6 # Oppgave 1.2
7 print(f"Med en bruttoinntekt på {inntekt} kr og en skattesats på {skattesats}
8      *100}%, blir skattetrekket {skattetrekk} kr.")
```

Fasit Oppgave 1.3: Forklar koden

Koden printer ut 40000. I programmering fungerer likhetstegnet (=) som en tilordningsoperator (en pil mot venstre), ikke som et matematisk likhetstegn. På linje to tar Python verdien som allerede ligger i variabelen (50000), trekker fra 10000, og overskriver den gamle verdien i minnet med den nye verdien (40000).

Del 2: Funksjoner

```
1 # Oppgave 2.1
2 def beregn_naaverdi(fremtidig_belop, rente, aar):
3     pv = fremtidig_belop / ((1 + rente)**aar)
4     return pv
5
6 # Oppgave 2.2
7 naaverdi = beregn_naaverdi(150000, 0.08, 5)
8 print(f"Nåverdien er {round(naaverdi, 2)} kr.")
```

Fasit Oppgave 2.3: Forklar koden

Programmet krasjer fordi funksjonen bruker `print(mva)` i stedet for `return mva`. `print` viser bare tallet på skjermen for oss mennesker, mens funksjonen faktisk leverer `None` ("ingenting") tilbake til datamaskinen. Når Python prøver å regne ut `1000 + None`, utløses en `TypeError` fordi man ikke kan addere et heltall og en `NoneType`. For å rette dette må `print(mva)` byttes ut med `return mva`.

Del 3: Datastrukturer og NumPy

```
1 # Oppgave 3.1
2 aksjer = {"EQNR": 290, "DNB": 210, "YAR": 350}
3 aksjer["MOWI"] = 190
4 print(aksjer)
5
6 # Oppgave 3.2
7 kontantstrommer = [-1000, 200, 300, 400, 500]
8 investering = kontantstrommer[0]
9 print(f"Opprinnelig investering: {investering}")
10 # Forklaring Tuple vs Liste: En Tuple brukes når vi har data
11 # som IKKE skal kunne endres i løpet av programmet (immutable).
12 # F.eks. historiske børskurser, GPS-koordinater eller konstanter.
13
14 # Oppgave 3.3
15 import numpy as np
16
17 cf_array = np.array(kontantstrommer)
18 # Merk: For å kun justere positive tall kreves filtrering med boolsk maske,
```

```

19 # men for enkelhets skyld kan vi også bare multiplisere hele arrayet:
20 justert_array = cf_array * 1.05
21 print(justert_array)

```

Fasit Oppgave 3.4: Forklar koden

I Kodeblokk A multipliseres en standard Python-liste. Da vil Python bare repetere innholdet i listen, slik at resultatet blir [100, 200, 300, 100, 200, 300]. I Kodeblokk B brukes et NumPy-array. NumPy utfører matematikk "vektorisert" (element for element), slik at resultatet faktisk blir matematiske utregninger: [200, 400, 600]. Dette er hovedgrunnen til at vi bruker NumPy for finansielle og statistiske beregninger.

Del 4: Pandas og databehandling

```

1 # Oppgave 4.1
2 import pandas as pd
3
4 # Antar at filen ligger i samme mappe som koden.
5 url = "Median_totalpris_boliger_Oslo.csv"
6 df = pd.read_csv(url)
7 print(df.head())
8
9 # Oppgave 4.2
10 print("Dimensjoner (rader, kolonner):", df.shape)
11 print("Manglende verdier:\n", df.isna().sum())
12
13 # Oppgave 4.3
14 df_2023 = df[["Geografi", "2023"]].set_index("Geografi")
15 print(df_2023.head())
16
17 # Oppgave 4.4
18 pris_frogner = df_2023.loc["Bydel Frogner", "2023"]
19 print(f"Median totalpris i Bydel Frogner var: {pris_frogner}")

```

Fasit Oppgave 4.5: Forklar koden

Linjen sorterer datasettet (df) basert på kolonnen "2023" i synkende rekkefølge (fra høyest til lavest, fordi ascending=False). Deretter bruker den .head(3) for å utelukkende hente ut de tre øverste radene. Resultatet blir en ny DataFrame som viser de tre bydelene i Oslo som hadde de dyreste boligene (medianpris) i 2023.

Del 5: Utfordring (Simulering)

```

1 # Oppgave 5.1
2 import numpy as np
3
4 kapital = 100000
5 aar = 0
6
7 # Løkken kjører så lenge kapitalen er under 200 000
8 while kapital < 200000:
9     # Trekker en tilfeldig avkastning mellom -2% og 8%
10    aarlig_avkastning = np.random.uniform(-0.02, 0.08)
11
12    # Oppdaterer kapitalen
13    kapital = kapital * (1 + aarlig_avkastning)
14
15    # Legger til et år på telleren
16    aar += 1
17
18 print(f"Det tok {aar} år før investeringen doblet seg til {round(kapital, 2)} kr .")

```